

Doing several things

Tasks and futures

Mark a function `task` when it is meant to run alongside other work. `spawn` starts it and gives back a future; `await` collects the answer.

```
use std.io

task fetch(id: Int) -> Int {
    give id * 100
}

fn main() {
    let a = spawn fetch(1)
    let b = spawn fetch(2)
    let c = spawn fetch(3)

    io.say("{await a} {await b} {await c}")
}
```

Spawning is cheap: nothing runs until something awaits it or the scheduler gets a turn. That keeps the common "start several, collect them all" pattern fast.

Channels

A channel is a queue that one part of the program writes to and another reads from.

```
use std.io
use std.thread

fn main() {
    let jobs = thread.channel(8)

    for n in 1..5 {
        jobs.send(n * n)
    }

    io.say("{jobs.len()} jobs waiting")
    while jobs.len() > 0 {
        io.say("took {jobs.recv() ?? 0}")
    }
}
```

`recv` gives nothing when the channel is empty and closed, so a loop can tell the difference between "wait" and "finished".

Locks and atomics

When two parts of the program really do share one value, guard it.

```
use std.io
use std.thread

fn main() {
    let guard = thread.lock()
    let counter = thread.counter(0)

    thread.acquire(guard)
    thread.atomic_add(counter, 5)
    thread.atomic_add(counter, 3)
    thread.release(guard)

    io.say("counter is {thread.atomic_add(counter, 0)}")
    io.say("{thread.cpus()} cores available")
}
```

What the runtime actually does

Threads are real operating system threads, but the interpreter holds one lock while it runs, handing it over every so often and whenever a task blocks on input, output or the network. That means input and output overlap properly and the program stays responsive, while two pieces of SPRFST code never run at the same instant. It is honest cooperative scheduling rather than parallel execution, and it keeps the memory model simple: you cannot see a half written value.

Exercise

Spawn five tasks that each square a number, collect them into a list, and print the total.